Task/Process Scheduling

Input :→ A set of tasks/processes that are in the ready state

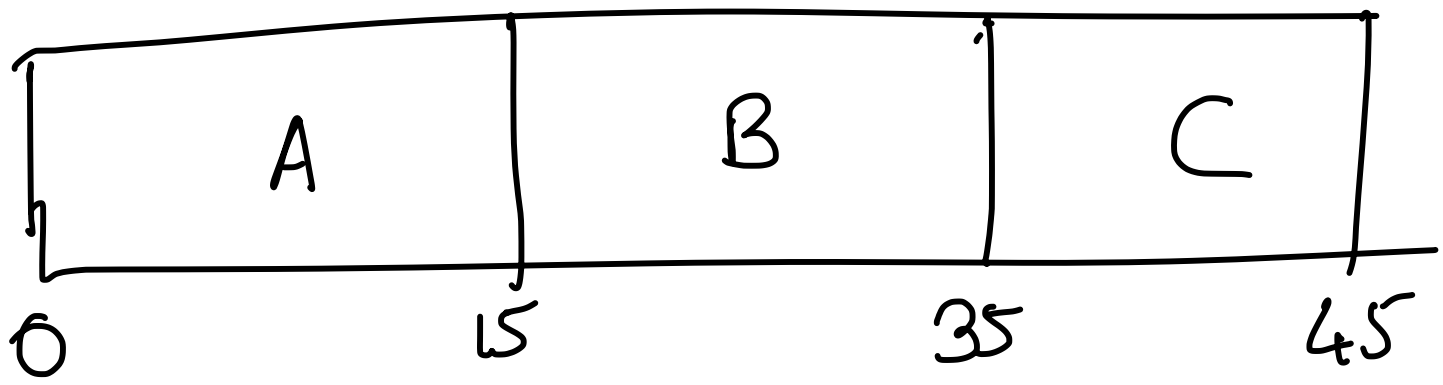
Output: → A schedule of processes, which is the same as selecting a process to send it to the running state.

First Come First Serve (FCFS)

A → 15 Time units, arrived at 0

B → 20 Time units, " " 1

C → 10 Time units, " " 2



Mean turnaround time = Avg time taken to complete the job since its beginning

$$T_{turnaround}(A) = 15$$

$$T_{turnaround}(B) = 35 - 1 = 34$$

$$T_{turnaround}(C) = 45 - 2 = 43$$

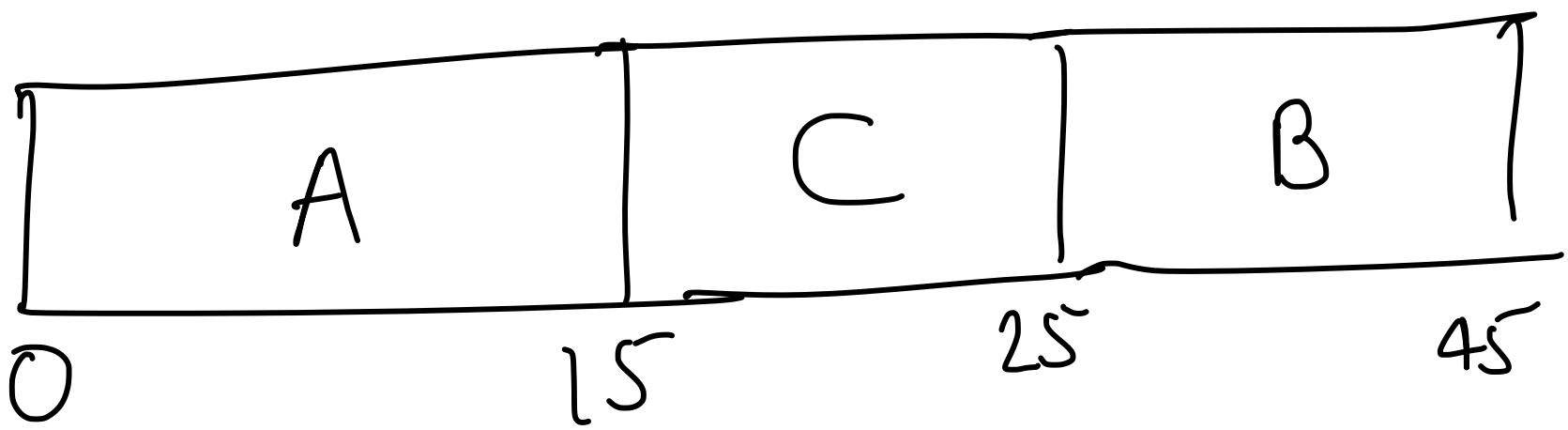
$$T_{turnaround}(Avg) = \frac{15 + 34 + 43}{3}$$

Average start time of FCFS can be large, which is unfair to many processes.

Convoy Effect: → Smaller processes that come in might have to wait for long time for a long process to finish.

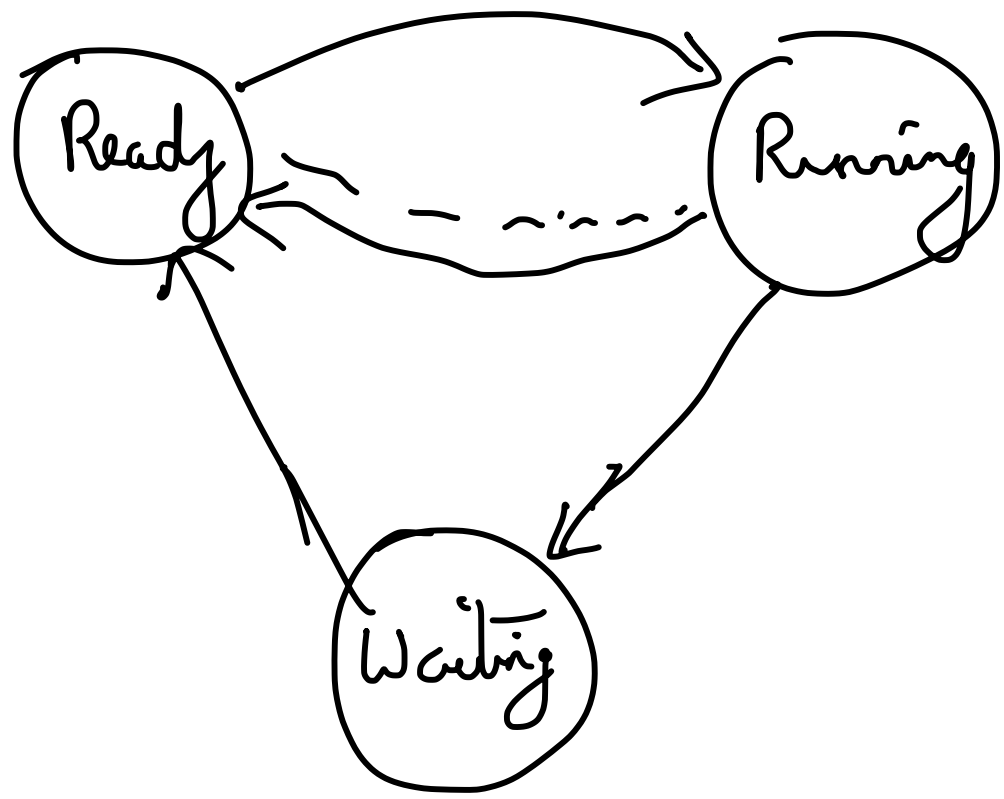
Shortest Job First

Schedule at each step the job that would take the shortest time to finish.

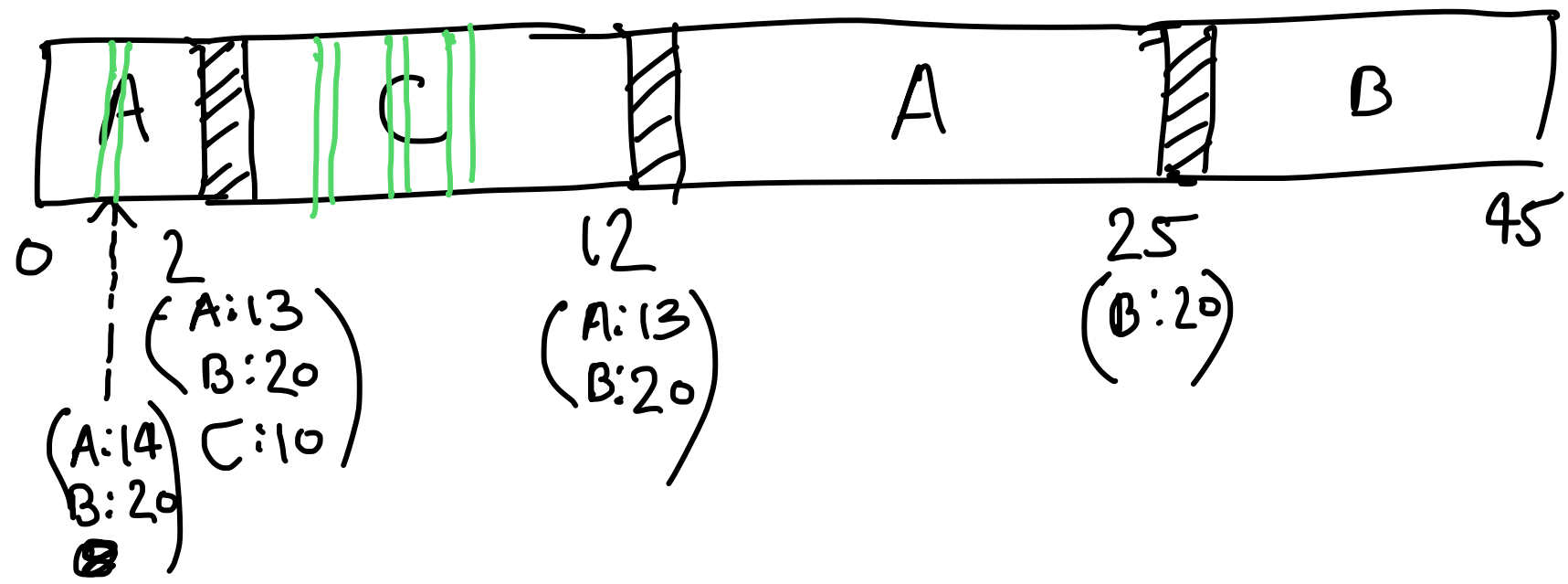


Starvation: → A process may not get access to the CPU for infinite time.

So far, the assumption is that the process cannot be moved back to the ready state from the running state; i.e. processes are non-preemptive in nature.

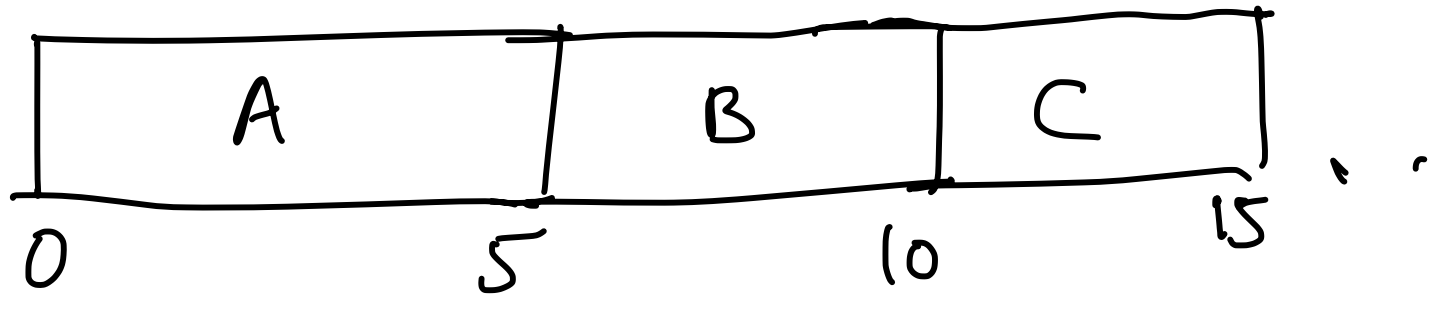


Preemptive version of SJF → [Shortest Job Remaining First]



Round-Robin Scheduling: → Decide some units of time called time quantum to Allocate each ready process t amount of time based on FCFS.

t = 5



Advantages: → (1) No process is kept waiting for too long.

(2) No need to have accurate estimates of process execution type.

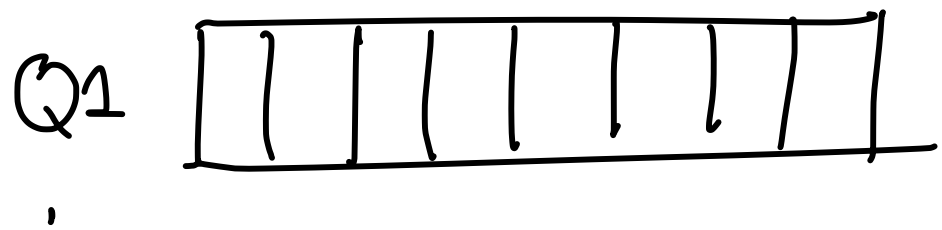
Disadvantages: → (1) The t value has to be determined experimentally.

(2) If t is large, then interactive jobs are not satisfied. If t is

small, then there are too many process switches.

(3) If t is known, then a process might be written in such a way that it uses I/O only just before its t finishes.

Multi level Feedback Queue



① Give higher priority to a process that is at a higher queue ⟹ If P1 is at a higher queue than P2, then schedule P1.

② Higher queue has a smaller time quanta than a lower queue.

③ Let every process start from Q1. If it uses the entire time quantum, then move it to lower queue for its next execution. If it does not use the entire time quantum, then keep its in the same queue.

④ Starvation-Free: → After some S amount of time, send all the lower-queue processes to the highest queue.

Scheduler performance is critically dependent on the heuristic values (time quanta and S).

The schedulers discussed today are fine for single processors. For multiprocessor scheduler, there needs to be proper handling of additional race conditions.